



Teaching LLMs with RL: Advancing Beyond Data Mimicry

Meet Mike Erlihson

1

Academic Background

Holds a PhD in Math, providing a strong theoretical foundation for his AI work.

2

Professional Leadership

Served as the Head of AI(currently at Drivenets), leading innovative advancements in several companies.

3

Influential Creator

A prominent scientific content creator and AI influencer with over 64K+ followers, sharing insights and fostering community growth.

4

Research Expertise

Has conducted extensive paper reviews, analyzing over 500 DL papers(DeepNightLearners project)

5

Community Engagement

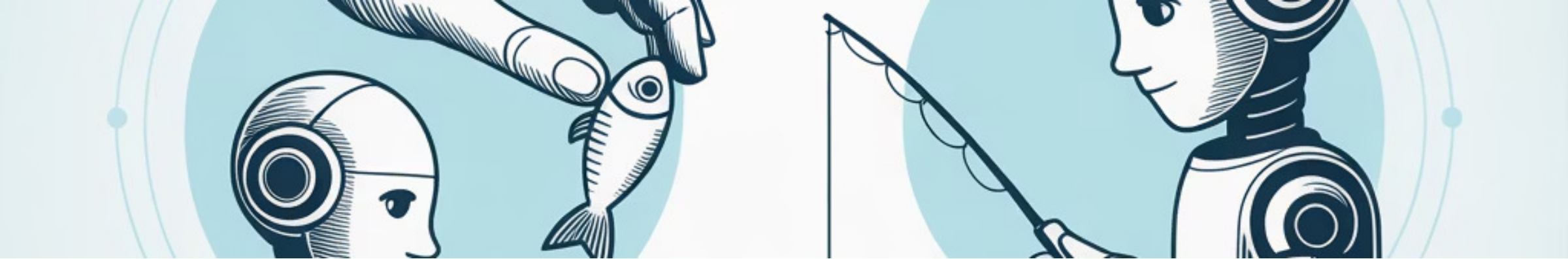
Hosts 2 popular podcasts, further engaging with the AI community, a co-author of the first ML/DL/AI book in Hebrew

What is Supervised Fine-Tuning(SFT)?

SFT trains LLMs with human-curated, labeled examples.

Teaches the LLM specific responses, aligning outputs with desired behaviors.





SFT: Giving a Fish, Not Teaching to Fish

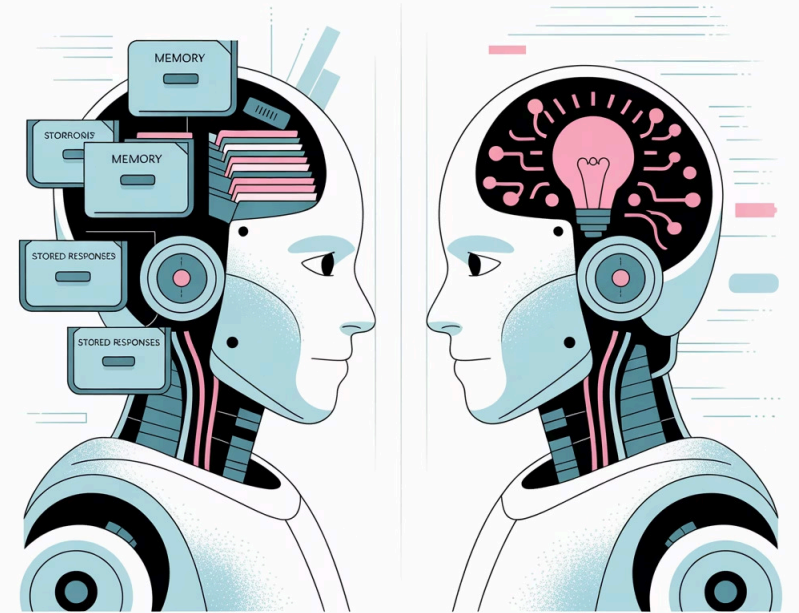
Provides models with specific answers, but has fundamental limitations hindering true learning & adaptability

Limited Relevance

Demonstrations may be useless, out-of-grasp, or irrelevant to the model's current knowledge gaps

Parroting instead of Understanding

LLMs often parrot memorized responses instead of learning true skills, limiting generalization & adaptability in novel situations.



High Quality Data Bottleneck

Relies heavily on the labeled data quality & quantity

Creating HQ datasets is time-consuming, costly & requires significant expertise.





SFT sucks - it is a FACT

Beyond Passive Data Mimicry: Memorization —> Understanding.

- SFT provides models with static "gold answers" while training maximizes the likelihood of these answers
- True learning demands iterative inference, enabling LLMs to generate responses & learn from evaluating their own outputs.



SFT: Static Gold Answers

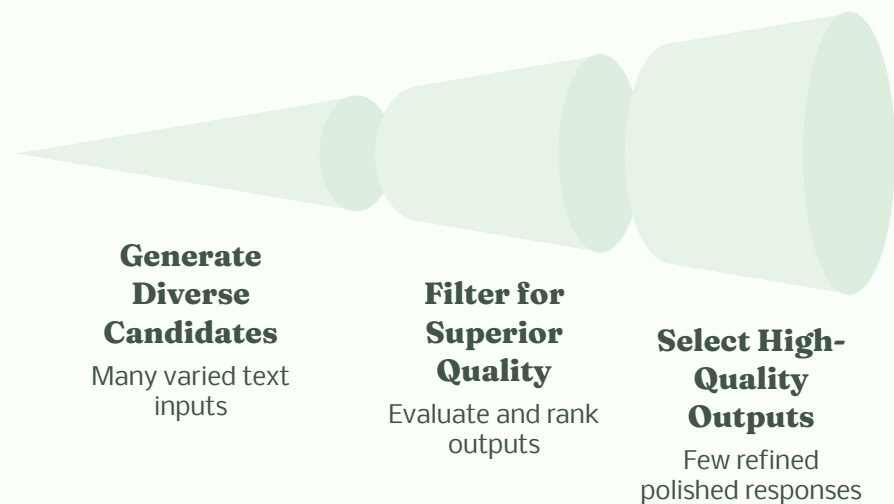
SFT provides models with static 'gold answers' while training maximizes the likelihood of these answers.

LANGUAGE MODEL



Using the Best LLM Outputs: Rejection Sampling

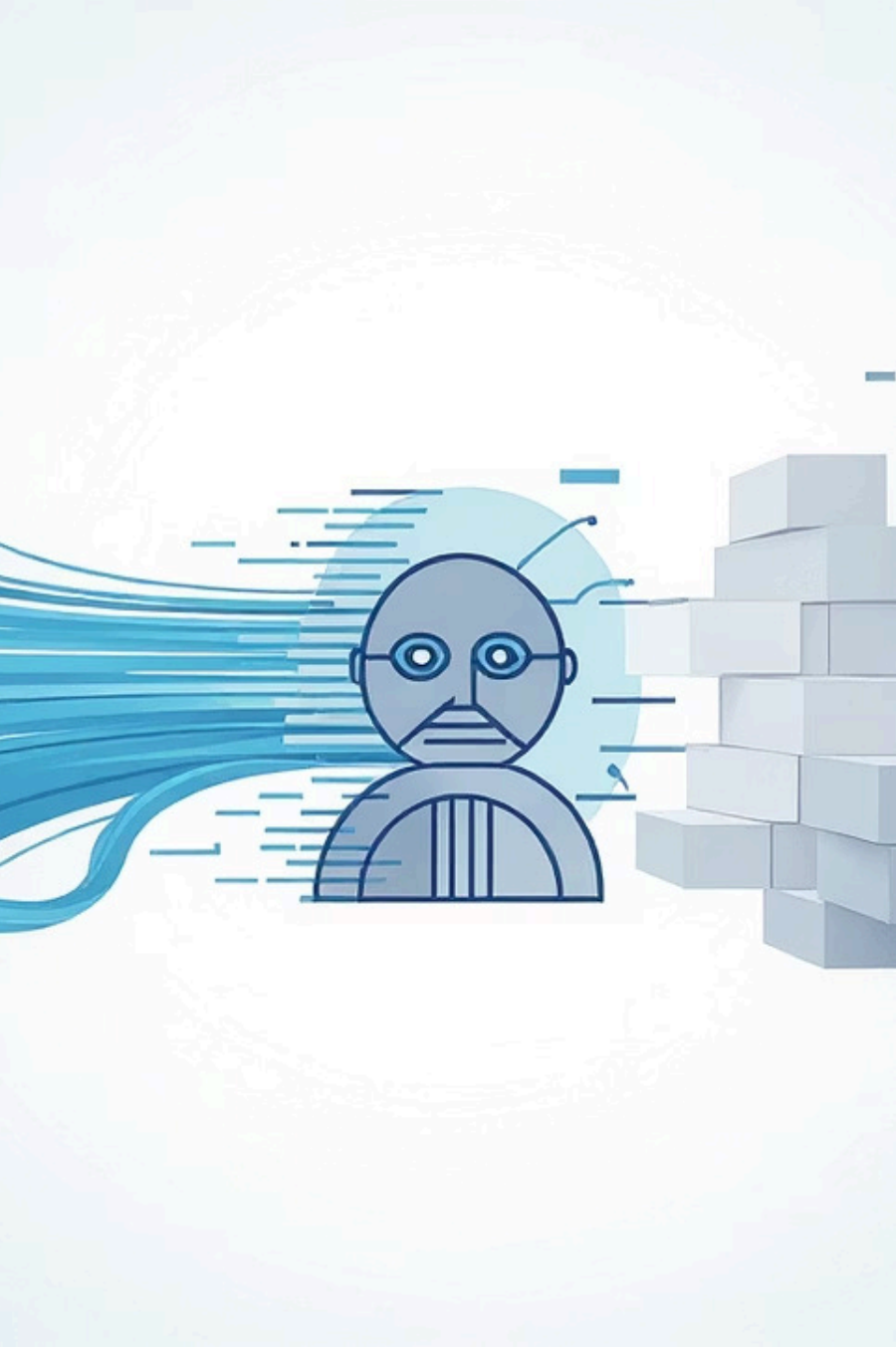
Leverage the model's own generation capabilities to build HQ responses via a rigorous selection process.



Prompt the LLM (with high temperature) to produce several candidate responses for a prompt.

These outputs are evaluated, with only the highest quality answers being selected, **by human annotators.**

Build a dynamic high-quality dataset changing with the model



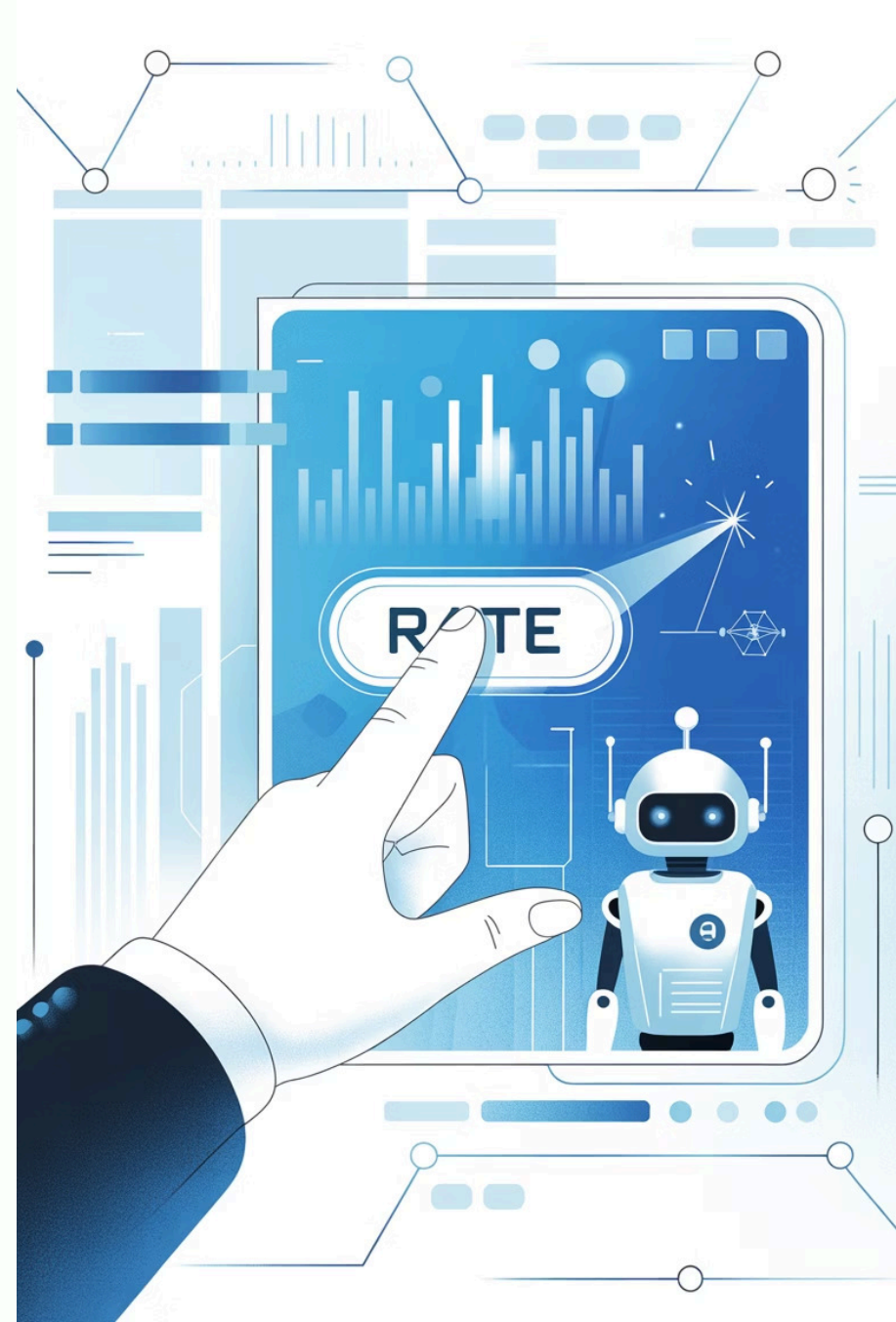
But the model changes...

Current offline data rating creates a bottleneck; models drift, making old data outdated.

Slow, manual process demands automation for continuous, online learning & adaptation.

From Offline Human Rating to the Reward Model(RM)

- RMs evaluate response quality, alignment, helpfulness etc **automatically.**
- Shifts from offline supervision to learned evaluation criteria.
- Enables continuous feedback rather than fixed examples.



Training the RM

Use human preference ratings to train a RM to evaluate LLM completions.



Human Feedback

Annotators provide ratings, training the RM to understand human preferences.



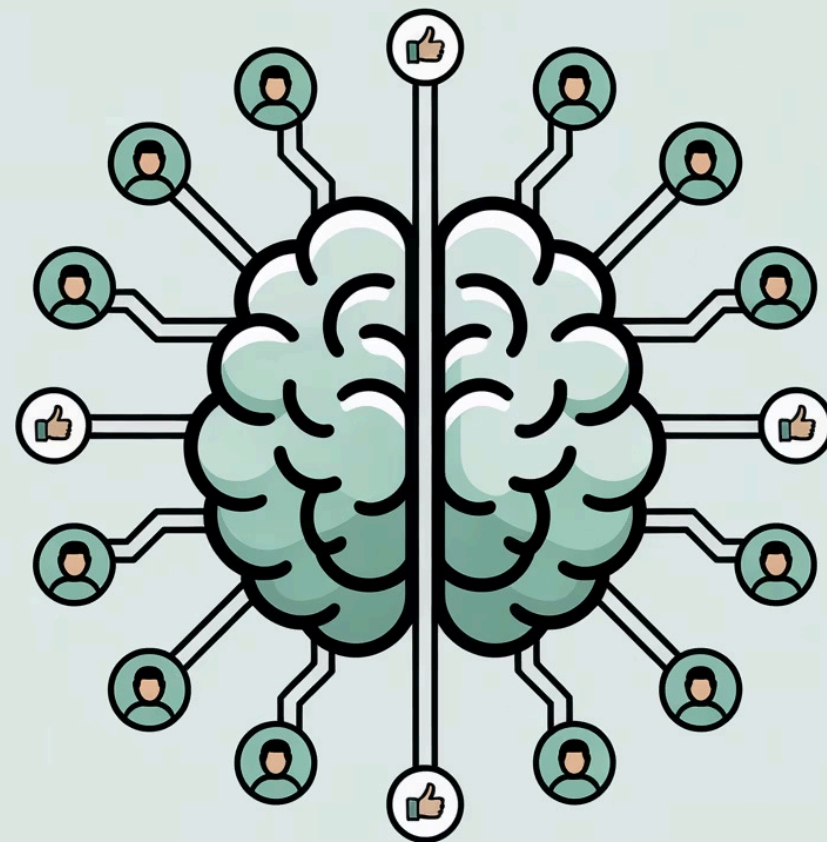
Automated Evaluation

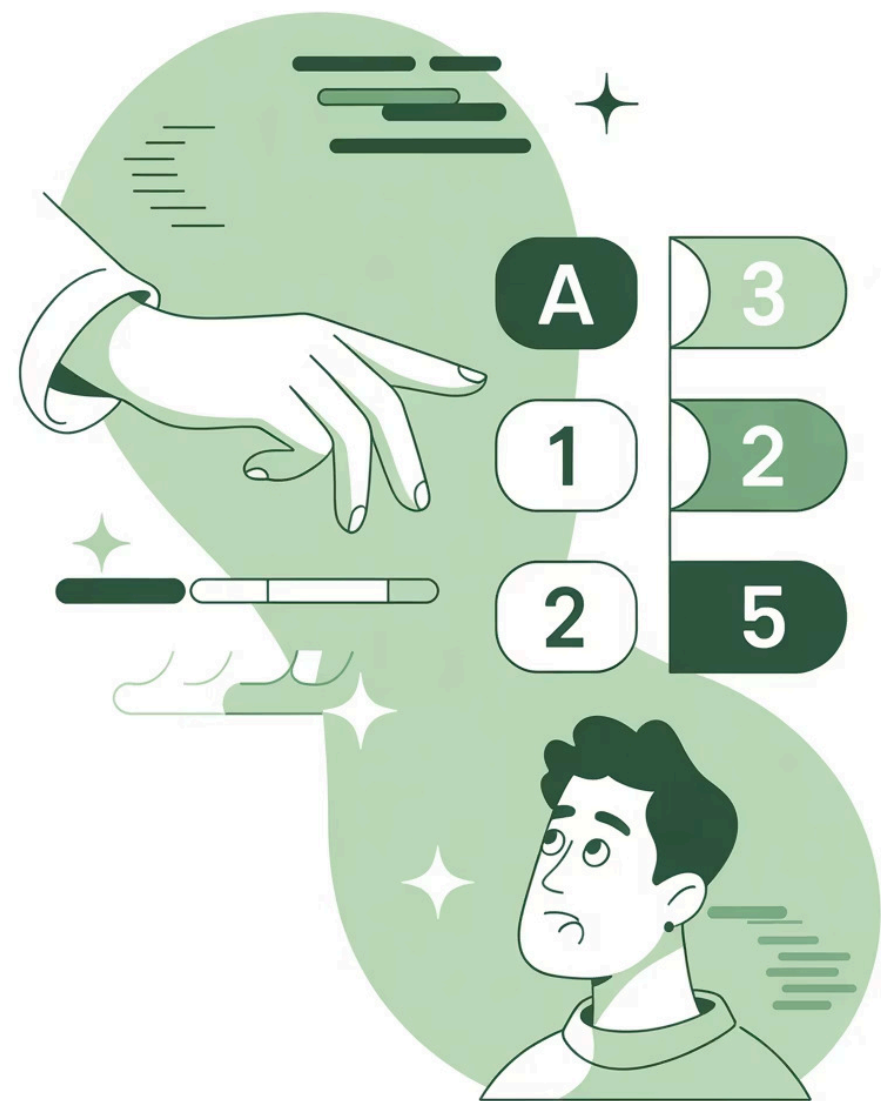
The RM assesses completion quality, removing dependency on external annotators.



Continuous Learning

Enables flexible, online adaptation & rejection sampling for ongoing improvement.





Preferences Over Absolute Ratings

Consistent quality ratings are challenging due to human subjectivity and calibration issues, especially with absolute scores (e.g., "3 out of 5").

Absolute Ratings

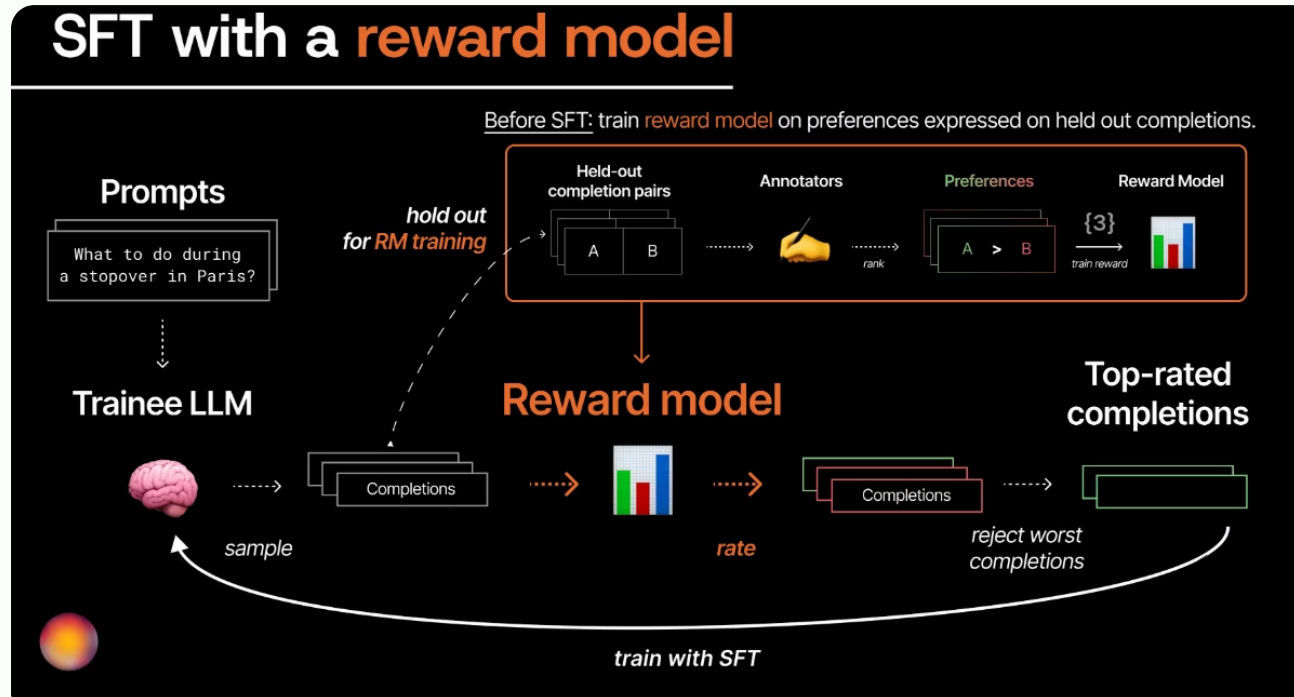
Humans struggle to apply consistent absolute scores across diverse samples and annotators.

Pairwise Preferences

Comparing two options ("A is better than B") is simpler and more reliable.

RMs can be trained on these pairwise preferences, yet still output a single, absolute reward score.

Rejection Sampling with Reward Model



{3} Reward model objective

$$\operatorname{argmax}_{\psi \in \Psi} \sum_{(x, y_+, y_-) \in \mathcal{D}_R} \left[\log(\sigma(R_\psi(x, y_+) - R_\psi(x, y_-))) \right]$$

log-sigmoid to scale difference in reward

$\psi \in \Psi$: reward model weights

$(x, y_+, y_-) \in \mathcal{D}_R$: sum over preference data

$R_\psi(x, y_+)$: reward model (good completion)

$R_\psi(x, y_-)$: reward model (bad completion)

Maximizes the difference in reward between good and bad completions.

Reinforcement Learning(RL): Empowering LLMs to Self- Improve

The RM provides a powerful feedback signal, quantifying the quality of LLM outputs.

→ **Beyond Selection**

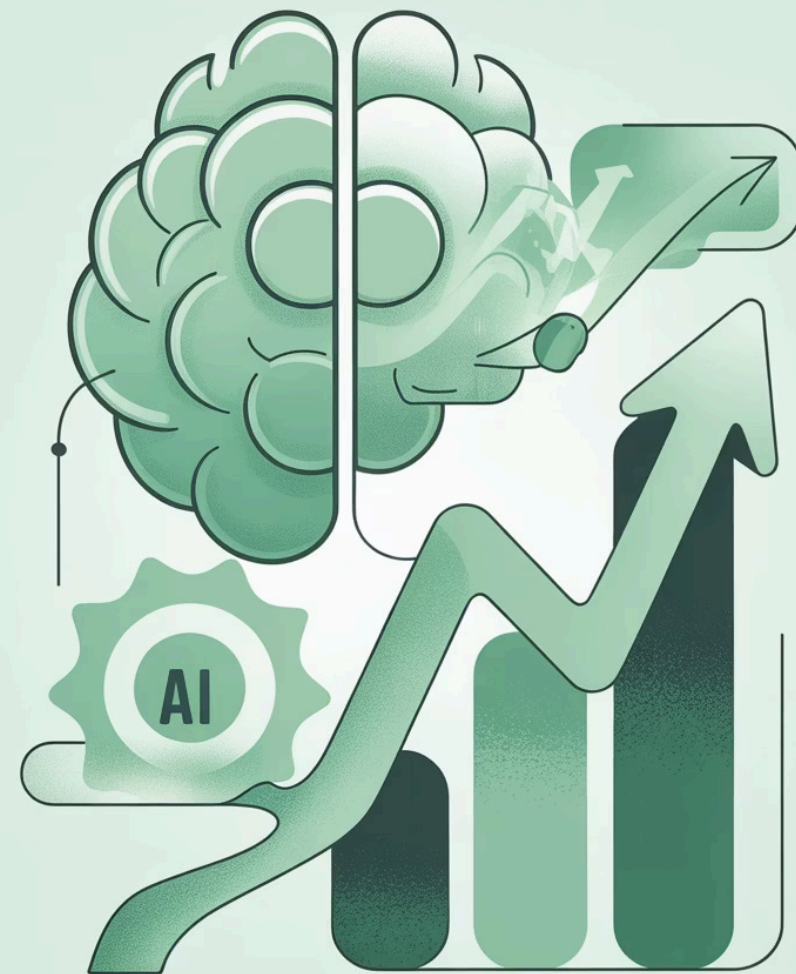
Instead of merely selecting the best responses, use the RM's signal to directly train the LLM.

→ **Active Learning**

Enables the model to actively learn from its own generations, optimizing its policy for higher-reward outputs.

→ **From Critic to Coach**

Transforms the RM from a passive critic into an active coach, guiding the LLM toward mastery.





Learning from All Responses

SFT with RM is trained on the top-rated completions: of the millions generated, only use a few 1000s are used

Can we leverage **all generated outputs** for learning?



Encourage Top-Rated Responses

Reinforce successful completions.



Discourage Low-Rated Responses

Learn from and avoid poor behaviors.



**The era of LLM training with RL has
officially started**

RL Terms Adapted for LLM Training Scenario



State

The current context, containing the initial prompt & all tokens generated up to now



Action

A new token generation, transitioning the model from one state to the next.



Policy Model

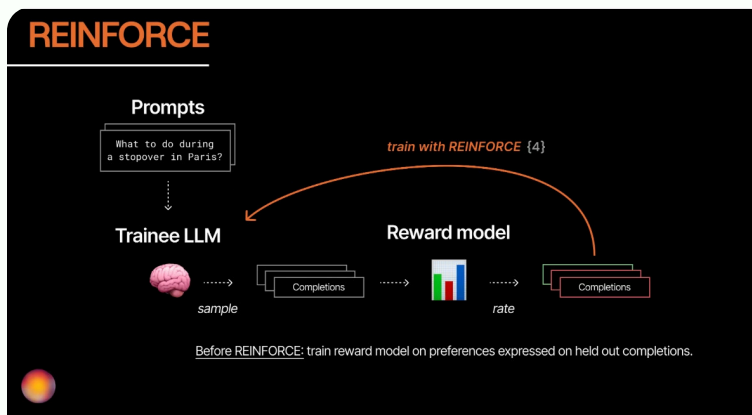
The trained LLM responsible for constructing a response (trajectory).



Value Model

Assesses the expected future reward from a state, assuming the current policy guides subsequent token generation (granular).

Classic REINFORCE Algo for LLM Fine-Tune



{4} REINFORCE objective

$$\operatorname{argmax}_{\theta \in \Theta} \sum_{x \in \mathcal{D}}^{\text{sum over all samples}} \underset{\substack{\text{reward} \\ \text{for the sample}}}{R(\mathbf{x})} \sum_{i=1}^n \overset{\text{trainee LLM}}{\log \pi_{\theta}(x_i | x_{<i})}$$

Maximises the likelihood of **good** completions (**positive reward**),
minimizes the likelihood of **bad** completions (**negative reward**).

The Credit Assignment Problem

REINFORCE assigns a single reward to an entire LLM response: doesn't distinguish b/w good & bad segments.

The "Paris Stopover" Dilemma

Question: What to do during a stopover in Paris?

- Sip on a decadent hot chocolate at Jacques Genin;
- Take a stroll around the White House lawn;
- Go for a walking tour of Montmartre.

The 1st and 3rd response are helpful, but the 2nd is irrelevant.

Unfair Reinforcement with completion-level rewards

The entire answer is or reinforced or penalized: unfair when parts are good & parts are bad.

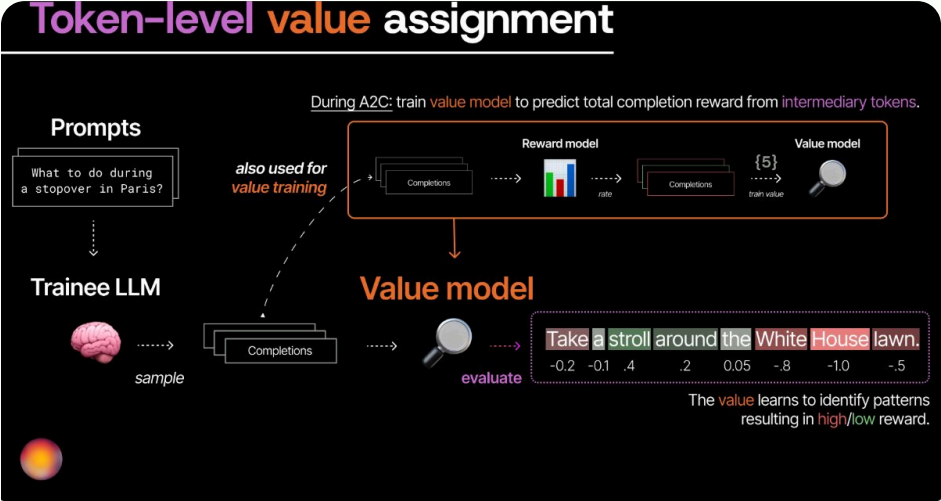
The LLM had $\frac{2}{3}$ of a reasonable answer, but the current approach can't partial credit.

We need to assign credit or blame at a **token-level** to understand where the model took a wrong turn.

Advantage Actor- Critic(A2C): Beyond Completion-level Rewards



Token-level Value Assignment



{5} Value model objective

$$\operatorname{argmin}_{\varphi \in \Phi} \sum_{x \in \mathcal{D}} \sum_{i=0}^n \left(V_{\varphi}(x_{<i}) - \sum_{j=0}^{n-i} R(x_{<i+j+1}) \right)^2$$

value
at the current state

reward
left to collect in following states

$\varphi \in \Phi$
value model weights

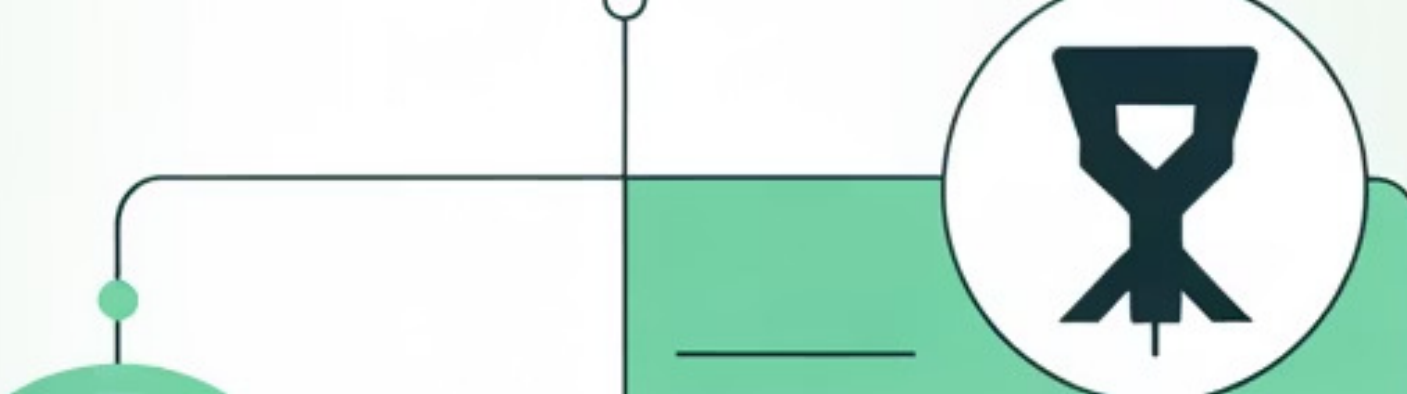
$x \in \mathcal{D}$
sum over states (tokens) in a sample x

$i=0$
sum over following states (remaining tokens to generate)

$n-i$
sum over following states (remaining tokens to generate)

$j=0$
sum over following states (remaining tokens to generate)

Minimizes the difference between value at current state and reward left to collect in following states.



Calculating Advantage: The Heart of A2C

Actor: The LLM

Our trainee LLM, responsible for generating sequences of tokens (actions) based on the current state.



Critic: The Value Model

Evaluates the long-term potential reward of each state, guiding the Actor towards optimal token choices.

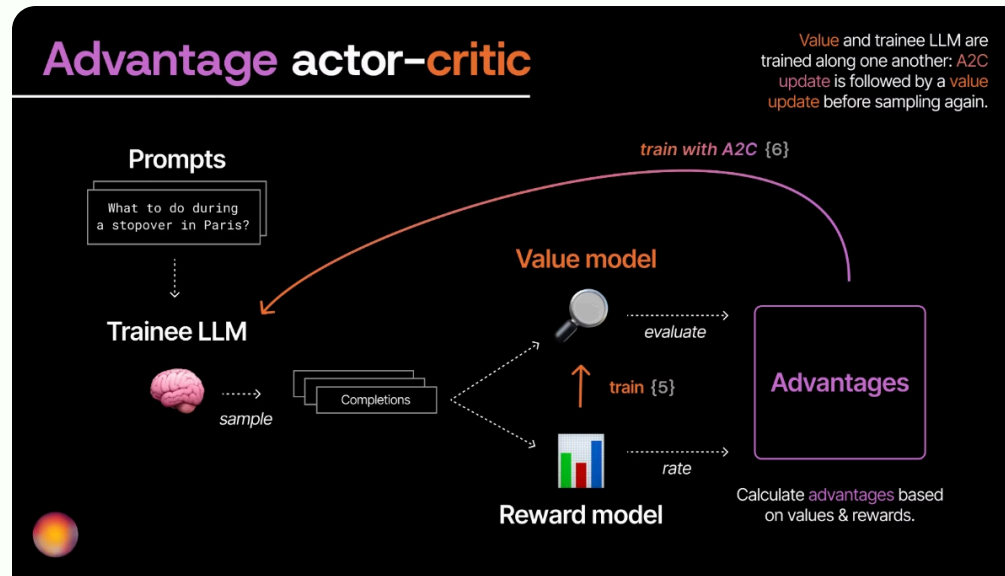
Action's Advantage

Measures the incremental value gained/lost by taking a specific action compared to the average expected outcome from the LLM's stochastic policy.

Generalized Advantage Estimation (GAE)

Combines both the **Reward Model's** feedback and the **Value Model's** predictions to accurately compute this advantage.

A2C Architecture & Objective



{6} Advantage actor-critic objective

advantage of generating a token given the context (action), according to the value model

$$\operatorname{argmax}_{\theta \in \Theta} \sum_{x \in \mathcal{D}} \sum_{i=1}^n A_{\varphi}(x_i, x_{<i}) \log \pi_{\theta}(x_i | x_{<i})$$

value model

trainee LLM (actor)

Maximizes the **likelihood of actions** (tokens) with a **positive advantage**, minimizes the **likelihood of actions** with a **negative advantage**.

How Value Models Enable Token-Level Learning

Statistical Attribution Across Trajectories

- 1** Over numerous completions, generic content's reward influence zeroes out. This highlights specific tokens consistently linked to positive/negative outcomes.

Pattern Recognition

- 2** The model identifies & generalizes patterns: actions consistently leading to positive/negative outcomes across completions.

Policy Training Integration(through Advantage)

- 3** A policy model integrates value predictions to adjust action dis(encouragement), based on leading to more valuable states.

A2C's Sampling Inefficiency Problem



Single-Use Trajectories

A2C processes each sampled response only once, leading to significant sampling inefficiency.

Valuable data from interactions isn't fully utilized.



Costly Sampling Process

Generating new responses through environment interactions is the most time-consuming & expensive part of training



Tool Use Bottlenecks

In envs requiring complex tool use, interactions become slower, making the single-use trajectory problem even more pronounced and hindering progress.



Policy Instability Risk

Reusing sampled trajectories by repeating A2C updates leads to policy divergence, where the agent's behavior becomes unstable & unpredictable (**distribution misalignment**)

Say Hello to Proximal Policy Optimization (PPO)

An RL algorithm striking a balance between powerful policy updates & maintaining training stability

1

Constrained Policy Updates

PPO employs a "clipped" objective function to ensure that each policy update remains within a defined proximity to the previous policy, preventing drastic shifts.

2

Improved Data Efficiency

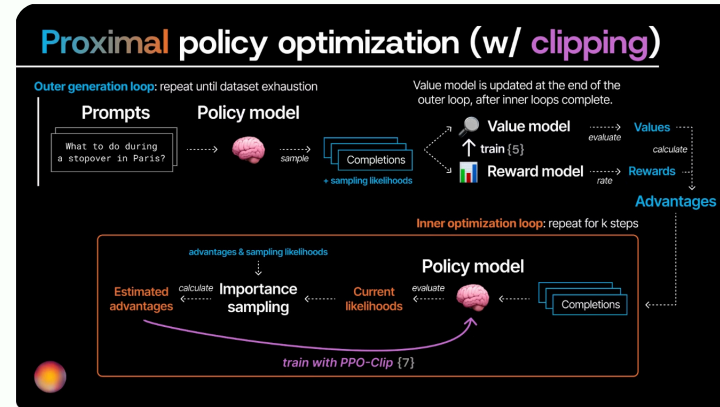
Unlike single-use trajectory methods, PPO enables multiple gradient updates on the same batch of sampled data, significantly reducing the need for new, costly interactions.

3

Enhanced Training Stability

By preventing excessively large and potentially destabilizing policy changes, PPO effectively mitigates the risk of policy divergence and ensures more reliable learning.

PPO with and w/o clipping



{7} Proximal policy optimization (w/ clipping)

$$r_{\theta} = \frac{\pi_{\theta}(x_i | x_{<i})}{\pi_{\theta_{old}}(x_i | x_{<i})}$$

current likelihood / sampling likelihood

$$\operatorname{argmax}_{\theta \in \Theta} \sum_{x \in \mathcal{D}} \sum_{i=1}^n \min \left(A_{\varphi} r_{\theta}, A_{\varphi} \operatorname{clip}(r_{\theta}, 1 - \varepsilon, 1 + \varepsilon) \right)$$

importance sampling ratio to estimate current advantage

advantages calculated with the value in the outer loop

clipping tolerance

minimum and clipping to constrain importance sampling ratio

Maximizes or minimizes the **likelihood of actions** given their **estimated advantage**, within the constraints of **clipping**.



Mitigating RM Over-Optimization: Reward Hacking

RMs can learn spurious correlations: policy model **will happily discover them**, leading to model outputting **word soup rather than natural language**.

Reason: RM is trained offline on samples from an older policy; Fresh samples would allow human annotators to catch this degradation (but unscalable)

1

Dynamic RM Updates

Split post-training into multiple PPO rounds. Between rounds, collect new completions from the latest policy to train a fresh RM

2

Reference Model Protection

Introduce a penalty for veering too far from the initial reference model (the one used to generate the RM's training data), safeguarding policy stability.

PPO with clipping and KL Divergence

Outer loop: repeat until dataset is exhausted

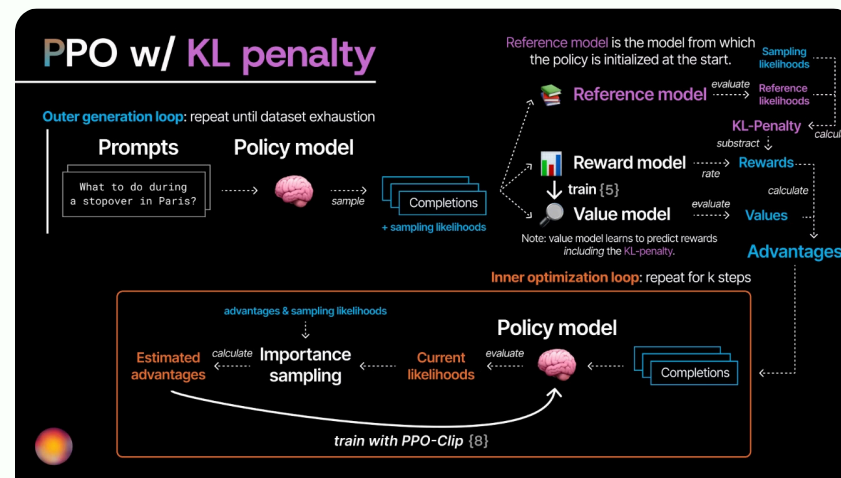
Sample *completions* with the policy model for several prompts

Obtain the reference likelihoods

Compute the token-wise *KL penalty* using reference and sampling likelihoods.

Evaluate completions with the RM & deduce the KL-penalty

Compute completions' values with the value model



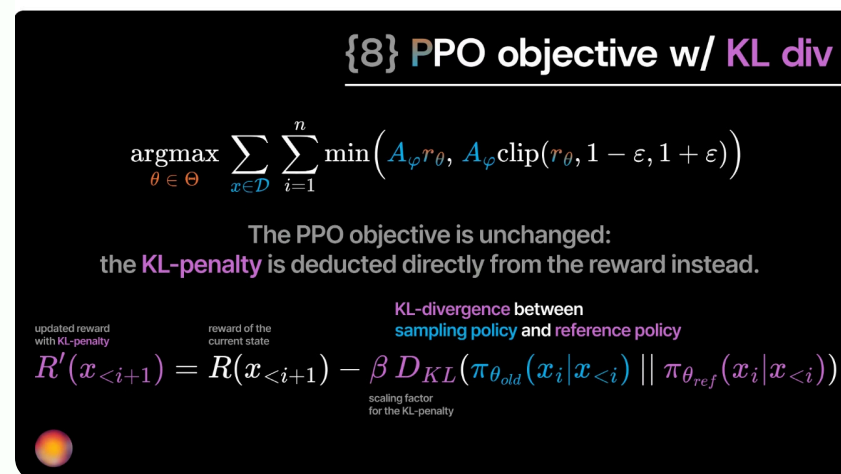
Inner optimization loop: repeat for k steps

Compute current completion likelihoods using the current policy model

Compute the importance sampling ratio between sampling & current likelihoods, (1 for the 1st step)

Update the policy model using the PPO objective

Fit value model using rewards & values.



PPO's Memory-Intensive Endeavor



Current Policy Model

The LLM being trained & optimized to generate responses



Reference Policy Model

A frozen version of the policy model: to calculate KL divergence & prevent drastic policy shifts.



Reward Model

Evaluates the quality of generated responses,



Value Model

Estimates the expected cumulative future reward from a given state, used to compute the advantage function in A2C.

- ❑ 4 **distinct models** demand significant computational resources, leading to high memory consumption & substantially increasing training costs.

Direct Preference Optimization (DPO): Your LLM is secretly a RM

Core Mechanism

Direct optimization based on human preferences, implicitly learning a reward function without an explicit RM

Key Advantages

Simplifies alignment, less computational overhead, memory & faster training.

Limitations

Requires high-quality human preference data. Suboptimal for extremely fine-grained reward signals where PPO's explicit models excel.

Group Relative Policy Optimization (GRPO)

Use relative rewards among a group of responses, instead of a separate value model.



Generate Diverse Responses

For each prompt, the model generates multiple potential completions, creating a group of candidates.



Compare Rewards Relatively

Responses are judged their quality relative to other completions within the same generated group.



Update Based on Preference

The model is updated to increase the likelihood of producing higher-scoring responses from the group, while maintaining proximity to the original policy.

Simplified Architecture

Removes the need for a separate value model, streamlining the training process and making it faster.

Enhanced Stability & Efficiency

More stable & efficient learning, beneficial for complex tasks like reasoning/code generation.

RL with Verifiable Rewards (RLVR)

The models are trained using objective, deterministic feedback, unlike subjective RLHF.

Objective Feedback

- 1** A "verifier" (unit test, script, math problem checker) examines the output against predefined criteria, ensuring deterministic evaluation.

Binary Reward

- 2** Simple and precise feedback: a "yes" or "no" for correctness; prevents over-optimization on ambiguous/subjective goals.

Skill Training

- 3** The model learns to generate outputs consistently receiving positive rewards, fostering reasoning & problem-solving skills in verifiable domains like math & coding.



LLM Training with RL Zoo: A Small Part



Questions & Answers

